

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Tesi di Laurea Magistrale in
High Performance Computing Engineering –
Ingegneria del Calcolo ad Alte Prestazioni

Contention-Aware Expert Parallel Load Balancing in Mixture-of-Experts Serving in vLLM

Author:
WU CAO

Student ID:
11036000

Advisor:
Prof. Qiaolun Zhang

Co-advisor:
Prof. Massimo Tornatore

Academic Year:
2026–2027

Abstract

Mixture-of-Experts (MoE) language models increase parameter capacity while activating only a subset of experts for each token. At inference time, expert parallelism distributes those experts across accelerators. The resulting load is input dependent: a small group of experts can become disproportionately popular, leaving some ranks overloaded while others are underutilized. Expert Parallel Load Balancing (EPLB) addresses this imbalance by periodically changing expert placement and migrating expert weights. In asynchronous EPLB, inference continues during migration; consequently, migration traffic and latency-sensitive inference traffic share the same network resources.

This thesis studies the network interference created when many point-to-point expert migrations are submitted simultaneously. It first formulates migration scheduling as an integer linear program (ILP) that minimizes the worst modeled inference time under link contention, then minimizes the number of active batches without degrading that primary optimum. The formulation exposes the central trade-off: spreading transfers over several batches reduces peak link load but can lengthen the migration itself. Based on this insight, a lightweight online scheduler is designed for vLLM. Expert transfers are represented as a rank communication multigraph, coalesced by directed rank pair, and greedily grouped into batches in which every rank participates in at most one directed flow. Schedules for all MoE layers are prepared once per rebalance cycle, while weight transfer and activation remain layer by layer. The implementation is opt-in, applies only to asynchronous EPLB, preserves the original synchronous path, and was merged upstream in vLLM pull request #52641.

The system is evaluated on four nodes connected by 10 Gb/s Ethernet, each with one Quadro RTX 6000 GPU, using Qwen3-30B-A3B-Instruct-2507, Ray, and the NIXL communication backend. In a controlled 500-request A/B benchmark, batching increased output throughput by 2.28%, reduced median and P99 time-per-output-token by 1.94% and 0.87%, and reduced median and P99 end-to-end latency by 3.18% and 1.50%. P99 aggregate NIC traffic on the head node decreased by 10.24%. NVTX-only Nsight Systems traces measured 34 cycle-wide scheduler invocations per rank; the largest cumulative scheduling cost was 302.081 ms, equal to 0.285% of the 105.932 s serving-time reduction

observed in the independent A/B run. These measurements show that, for the evaluated configuration, conservative rank-disjoint batching can trade a small amount of CPU scheduling work and migration serialization for lower network bursts and improved serving performance.

Keywords: mixture of experts, expert parallelism, load balancing, expert migration, network contention, vLLM, NIXL

Abstract in lingua italiana

I modelli linguistici Mixture-of-Experts (MoE) aumentano la capacità in termini di parametri attivando, per ciascun token, solo un sottoinsieme degli esperti. Durante l'inferenza, l'expert parallelism distribuisce tali esperti tra più acceleratori. Il carico risultante dipende dall'input: alcuni esperti possono diventare molto più richiesti di altri, sovraccaricando determinati rank e lasciandone altri sottoutilizzati. L'Expert Parallel Load Balancing (EPLB) riduce questo squilibrio modificando periodicamente il posizionamento degli esperti e migrandone i pesi. Nella modalità asincrona, l'inferenza continua durante la migrazione; il traffico di migrazione e quello dell'inferenza condividono quindi le stesse risorse di rete.

Questa tesi analizza l'interferenza di rete causata dall'invio simultaneo di molte migrazioni punto-punto. Il problema viene inizialmente formulato come un programma lineare intero (ILP), con l'obiettivo di minimizzare il peggior tempo di inferenza modellato in presenza di contesa sui link; un secondo obiettivo minimizza il numero di batch senza peggiorare l'ottimo primario. La formulazione mette in evidenza il compromesso fondamentale: distribuire i trasferimenti su più batch riduce il picco di traffico, ma può aumentare la durata complessiva della migrazione. Sulla base di questa osservazione è stato progettato uno scheduler online leggero per vLLM. I trasferimenti sono rappresentati come un multigrafo di comunicazione tra rank, aggregati per coppia diretta e raggruppati con un algoritmo greedy in batch nei quali ogni rank partecipa al massimo a un flusso diretto. Gli schedule di tutti i layer MoE sono calcolati una sola volta per ciclo di riequilibrio, mentre il trasferimento e l'attivazione restano sequenziali per layer. L'implementazione è opzionale, riguarda esclusivamente EPLB asincrono e mantiene invariato il percorso sincrono; è stata integrata upstream in vLLM tramite la pull request #52641.

La valutazione usa quattro nodi connessi tramite Ethernet a 10 Gb/s, ciascuno dotato di una GPU Quadro RTX 6000, con il modello Qwen3-30B-A3B-Instruct-2507, Ray e il backend NIXL. In un benchmark A/B controllato di 500 richieste, il batching aumenta il throughput di output del 2.28%, riduce il tempo per token di output P50/P99 dell'1.94%/0.87% e la latenza end-to-end P50/P99 del 3.18%/1.50%. Il P99 del traffico NIC aggregato sul nodo principale diminuisce del 10.24%. Le tracce NVTX raccolte con Nsight

Systems misurano 34 invocazioni dello scheduler per rank; il costo cumulativo massimo è 302.081 ms, pari allo 0.285% dei 105.932 s risparmiati nel benchmark A/B indipendente. Per la configurazione studiata, i risultati mostrano che un batching conservativo e privo di conflitti tra rank può ridurre i picchi di rete e migliorare le prestazioni di serving con un costo CPU contenuto.

Parole chiave: mixture of experts, expert parallelism, bilanciamento del carico, migrazione degli esperti, contesa di rete, vLLM, NIXL

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

List of Figures **ix**

List of Tables **xi**

Acronyms **xiii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Research questions . . . 2
1.3 Contributions . . . 2
1.4 Scope and claims . . . 3
1.5 Thesis structure . . . 4

2 Background and Related Work **5**
2.1 Autoregressive language-model serving . . . 5
2.2 Sparse mixture-of-experts models . . . 5
2.3 Expert parallelism . . . 6
2.4 Expert Parallel Load Balancing in vLLM . . . 7
2.5 NIXL communication . . . 7
2.6 Related Work . . . 8
2.6.1 MoE communication and load balancing . . . 8
2.6.2 LLM serving interference and runtime movement . . . 9
2.6.3 Collective-flow scheduling . . . 9
2.6.4 Graph scheduling and expert migration . . . 9

2.6.5 Positioning of this thesis 10

3 Problem Statement and Formulation 11
3.1 Migration instructions and flows 11
3.2 Problem inputs and output 11
3.3 Network model 12
3.4 Constrained integer linear program 12
3.5 Migration-time diagnostic 14
3.6 Illustrative hotspot instance 14
3.7 Why the ILP is not used online 16

4 Methodology and System Design 17
4.1 Design requirements 17
4.2 From placements to remote flows 17
4.3 Greedy rank-disjoint grouping 18
4.4 Correctness invariants 19
4.5 Complexity 20
4.6 Cycle-wide precomputation, layer-wise transfer 20
4.7 Batched execution 20
4.8 Configuration and failure containment 21
4.9 Upstream integration 21

5 Experimental Evaluation 23
5.1 Methodology 23
5.1.1 Evaluation goals 23
5.1.2 Hardware and software environment 23
5.1.3 Model and EPLB configuration 24
5.1.4 Workload 25
5.1.5 Baseline and proposed condition 25
5.1.6 A/B procedure and controls 25
5.1.7 Serving metrics 26
5.1.8 NIC measurement 26
5.1.9 Scheduler profiling 27
5.1.10 Data provenance and reproducibility 27
5.2 Results and Discussion 28
5.2.1 End-to-end serving results 28
5.2.2 Network traffic 29
5.2.3 Scheduler profiling 29

5.2.4 Answering the research questions 32
5.2.5 Discussion 32
5.2.6 Threats to validity 33

6 Conclusions **35**

Bibliography **37**

A Reproduction Commands **41**
A.1 Environment and server 41
A.2 Warm-up and formal workload 41
A.3 NVTX-only Nsight capture 42
A.4 Unit tests 42

B Metric Definitions and Supplementary Results **43**
B.1 Latency definitions 43
B.2 Complete improvement table 43
B.3 Artifact map 44

Acknowledgements **47**

List of Figures

2.1 Simplified asynchronous EPLB control loop. The contribution of this thesis is the migration-scheduling block and its execution path. 8

3.1 Illustrative hotspot topology and migration schedules before and after contention-aware batching. The unbatched diagram remains a directed rank graph; only its layout is vertical. 15

5.1 Relative improvement with batching enabled in the clean A/B run. Positive values indicate higher throughput or lower latency. 28
5.2 Head-node aggregate NIC traffic sampled once per second during the two independent runs. Batching reduces the high tail of observed traffic. 30
5.3 Largest measured cumulative scheduler time versus serving time saved. The linear axis intentionally preserves the scale difference. 31

List of Tables

3.1 Result of the illustrative ILP instance. Values are normalized. 16

5.1 Experimental environment. 24
5.2 EPLB parameters common to the A/B experiment. 24
5.3 End-to-end serving results. Lower is better for all latency columns; higher is better for throughput. Each condition is one run. 30
5.4 NVTX elapsed time for cycle-wide scheduler calls. 31

B.1 Final clean A/B values and descriptive relative improvements. 44
B.2 Final artifact locations relative to `benchmarks/eplb_rebalance`. 44

Acronyms

EP expert parallelism. 1, 6

EPLB Expert Parallel Load Balancing. 1

ILP integer linear program. 3

MoE Mixture of Experts. 1

NIXL NVIDIA Inference Xfer Library. 7

TPOT time per output token. 2, 5

TTFT time to first token. 2, 5

1 | Introduction

1.1. Motivation

Large language model serving is a systems problem as much as a model problem. An online service must keep expensive accelerators occupied, admit requests with different prompt and generation lengths, manage a dynamically growing key–value cache, and coordinate communication across devices. vLLM addresses several of these concerns through continuous batching and PagedAttention [11]. The adoption of sparse Mixture of Experts (MoE) models introduces a further challenge: the compute and communication performed for a token depend on which experts the router selects.

An MoE layer contains many feed-forward expert networks but activates only a small subset for each token. This conditional computation increases model capacity without making the arithmetic cost proportional to the total number of parameters [6, 20]. It also makes the runtime load data dependent. Tokens in a batch do not necessarily select experts uniformly, and the popularity distribution can shift as the request mix changes. When experts are sharded across devices with expert parallelism (EP), an overloaded expert becomes an overloaded rank. Faster ranks then wait at collective communication boundaries, so one local imbalance can reduce cluster-wide throughput.

Expert Parallel Load Balancing (EPLB) mitigates this problem by observing expert usage and periodically computing a new physical placement. Popular experts can be replicated and the mapping between logical experts and physical slots can be changed. Applying a new placement, however, requires copying expert weights between ranks. These objects are much larger than control messages and the transfers use the same host and network resources as foreground inference. This thesis focuses on a specific but consequential phase of EPLB: scheduling those expert migrations.

The original execution path posts the point-to-point operations for a layer as one group. This minimizes the number of explicit communication rounds, which is attractive if migration completion time is the only objective. During asynchronous EPLB, however, inference does not stop. A rank that simultaneously sends to and receives from several

peers can create a short traffic burst at its network interface. The burst may compete with token dispatch, result collection, tensor-parallel communication, or other inference-related traffic. Thus, the schedule that completes migration soonest is not necessarily the schedule that gives users the best latency while migration is in progress.

The distinction is especially important in a multi-node deployment whose network is slower than local accelerator memory. Expert migration is a background maintenance operation; the foreground objective is serving quality. The central hypothesis of this work is therefore that limiting the number of simultaneous migration peers per rank can reduce network bursts enough to improve foreground serving, even when doing so introduces additional migration rounds.

1.2. Research questions

The thesis is organized around four research questions.

RQ1: How should expert migration contention be represented? A useful model must distinguish expert count, source and destination ranks, network paths, and link capacities while remaining simple enough to explain the scheduler's behavior.

RQ2: What scheduling rule is suitable for the online path? An exact optimizer is valuable as a design tool, but a production scheduler must be deterministic, fast, dependency-light, and compatible with the existing replicated placement maps.

RQ3: Can the implementation preserve EPLB semantics? Batching must move every required expert exactly once, retain local-copy and replica semantics, respect the communicator lifecycle, and leave the legacy path unchanged unless the feature is explicitly enabled.

RQ4: Does reduced migration contention improve serving? The answer requires both user-visible metrics and direct network evidence. Throughput, time to first token (TTFT), time per output token (TPOT), and end-to-end latency are measured together with one-second NIC byte counters and scheduler profiling.

1.3. Contributions

This work makes the following contributions.

- It separates two objectives that can otherwise be conflated: minimizing migration completion time and minimizing the interference experienced by concurrent inference.
- It formulates contention-aware migration scheduling as an integer linear program (ILP). The model assigns coalesced directed rank-pair flows to batches, estimates routed link load, and first minimizes the worst modeled inference time. A second solve minimizes the number of batches without degrading that primary optimum. An illustrative eight-rank topology exposes the trade-off rather than claiming a calibrated latency predictor.
- It designs a production-oriented greedy scheduler. The problem is reduced to deterministic first-fit edge coloring of a communication graph. Multiple experts with the same directed source–destination pair are coalesced, independent rank pairs can progress concurrently, and every rank participates in at most one directed flow in a batch.
- It integrates the scheduler into asynchronous vLLM EPLB. Scheduling for all 48 MoE layers is invoked once per rebalance cycle, while transfers remain layer by layer. The feature is opt-in and synchronous EPLB retains the original path because inference is already paused in that mode.
- It evaluates the complete serving system and profiles the scheduler. The final contribution was reviewed and merged into upstream vLLM as pull request #52641 [2].

1.4. Scope and claims

The evaluation is deliberately narrow enough to be reproducible. It uses one four-node cluster, one model, one communication backend, one workload shape, and one A/B run per condition. This supports a case study and an engineering validation, not a universal performance claim. The experiment establishes that the proposed mechanism works and is beneficial under the measured configuration. It does not establish how the effect scales to modern datacenter GPUs, RDMA fabrics, different expert sizes, or more ranks.

The thesis also distinguishes artifacts by purpose. The ILP is an offline analytical baseline. The greedy algorithm is the deployed scheduler. Older experiments created during design iteration are not used for the final result; all reported E2E values come from one archived A/B directory, and all scheduler costs come from a separate archived profiling directory. This provenance is specified in section 5.1 and the appendices.

1.5. Thesis structure

Chapter 2 introduces sparse MoE inference, expert parallelism, vLLM EPLB, and asynchronous migration; its related-work section positions this thesis against MoE systems, coflow scheduling, and LLM serving research. Chapter 3 states the migration-scheduling problem and develops the offline ILP. Chapter 4 presents the production methodology, greedy scheduler, correctness properties, and vLLM integration. Sections 5.1 and 5.2 document the evaluation procedure and report the E2E, NIC, and profiling results within one experimental-evaluation chapter. Finally, chapter 6 summarizes the findings, limitations, and future work.

2 | Background and Related Work

2.1. Autoregressive language-model serving

An autoregressive language model generates one token at a time. Serving a request normally has two computational phases. During *prefill*, the model processes the prompt and constructs the key–value cache. During *decode*, the service repeatedly executes the model to produce one or a small number of new tokens. Prefill has substantial parallel work over prompt tokens, whereas decode is iterative and often sensitive to memory access, scheduling delay, and interference.

Modern serving engines batch requests whose sequences are at different positions. vLLM introduced PagedAttention to reduce fragmentation and enable flexible sharing of KV-cache blocks, thereby increasing the number of requests that can be served concurrently [11]. Continuous batching makes resource utilization higher, but also means that a background operation may affect multiple requests in different phases at the same time.

Four metrics are central in this thesis. Output throughput is the number of generated tokens per second over the measured interval. TTFT is the time from request submission until its first output token, so it includes admission, queueing, and prefill. TPOT averages the gaps between output tokens after the first token and is primarily associated with decode. End-to-end latency measures the complete request. Reporting both median and P99 values separates typical behavior from tail behavior. Precise definitions are given in appendix B.

2.2. Sparse mixture-of-experts models

The sparsely gated MoE layer was developed as a practical form of conditional computation [20]. Let x denote a token representation, E_i the ith expert, and $g_i(x)$ a router weight. A simplified MoE output is

$$y(x) = \sum_{i \in \mathrm{TopK}(g(x))} g_i(x) E_i(x). \tag{2.1}$$

Only the selected experts execute for x. This decouples total parameter capacity from active arithmetic. GShard demonstrated large-scale conditional computation with automatic sharding [12]; Switch Transformer simplified routing to one expert per token and highlighted both the scaling opportunity and the communication and load-balancing challenges [6]. GLaM and ST-MoE further demonstrated that sparse expert models can scale capacity while retaining practical training and transfer quality [5, 31]. Recent open models continue this direction. Qwen3 includes dense and MoE variants [27], while DeepSeek-V3 uses a large sparse architecture and an auxiliary-loss-free training-time balancing strategy [4].

The evaluated Qwen3-30B-A3B-Instruct-2507 checkpoint has 30.5 billion total parameters, approximately 3.3 billion activated parameters, 48 layers, 128 experts, and eight selected experts per token [18]. The gap between total and active parameters makes it a useful serving model: compute is sparse, but expert weights remain substantial objects that must be placed and occasionally copied.

2.3. Expert parallelism

In EP, experts are distributed across ranks. A token is dispatched to the ranks holding its selected experts; expert outputs are then returned to the rank that continues the model computation. EP avoids replicating every expert on every device, but introduces all-to-all-style communication and a dependence on balanced rank workloads. DeepSpeed-MoE illustrates how expert, tensor, and data parallelism can be combined for large-scale MoE inference, while also showing that communication and model layout are central systems concerns [19].

Let $a_{k,e}$ be the number of token assignments to logical expert e in layer k over an observation window. If rank r hosts a set $H_{k,r}$, a simple load estimate is

$$U_{k,r} = \sum_{e \in H_{k,r}} a_{k,e}. \tag{2.2}$$

Even if training encourages globally balanced routing, a finite serving window can be skewed. Request language, task, domain, and generation state can all change which experts are selected. Moreover, the system cares about load per physical rank, not only load per

logical expert. Several moderately popular experts placed together can overload a rank.

2.4. Expert Parallel Load Balancing in vLLM

vLLM's EPLB records expert usage over a configurable window and periodically computes a new placement. Redundant experts add physical slots beyond the number required for one copy of each logical expert. The balancer can use those slots to replicate hot experts and distribute their assignments. vLLM exposes the observation window, rebalance interval, redundancy, asynchronous mode, policy, and migration communicator through `EPLBConfig` [24].

Placement is represented by replicated logical-to-physical and physical-to-logical mappings for every MoE layer. Once a new map is available, each rank can independently derive which logical experts it loses, keeps, or gains. Local rearrangements copy weights between slots on the same rank. Remote gains require a source rank that already owns the logical expert and a point-to-point weight transfer.

Synchronous and asynchronous EPLB have different performance objectives. A synchronous rearrangement pauses inference. In that setting, extra sequential communication rounds directly extend the pause, so quickly completing the migration is the natural goal. Asynchronous EPLB moves experts in a background worker while inference continues. It transfers one layer and coordinates its activation with forward progress before proceeding. Here, the migration has more time, but interference with foreground work matters. This thesis changes only the asynchronous path and retains the synchronous behavior.

Figure 2.1 summarizes this control loop. Expert-load measurements produce a new placement, the migration scheduler orders the required weight copies, and each layer is activated at a safe boundary. The dashed connection emphasizes the central problem: background transfers and foreground inference use the same network resources.

2.5. NIXL communication

NVIDIA Inference Xfer Library (NIXL) provides a transfer abstraction for distributed inference across heterogeneous memory and storage. A transfer agent manages registered memory, remote metadata, backend selection, and asynchronous point-to-point requests [15]. vLLM supports NIXL as an EPLB communicator and can also use Gloo or other backends depending on mode and configuration [23].

The final evaluation explicitly selects NIXL. This choice matters for both performance and

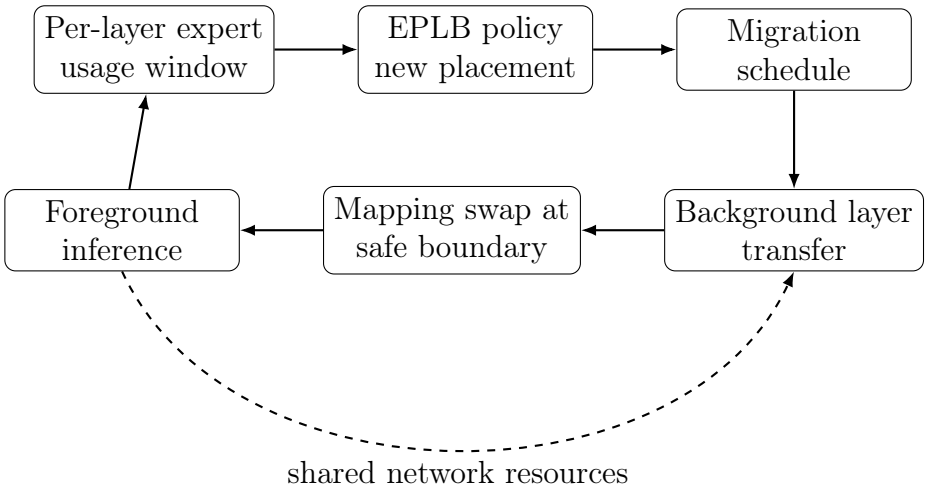


Figure 2.1: Simplified asynchronous EPLB control loop. The contribution of this thesis is the migration-scheduling block and its execution path.

correctness. In the implementation evaluated here, `execute()` drains the queued transfer operations and clears the transfer context. Consequently, every migration batch must restore the layer context, post only that batch's sends and receives, and call `execute()`. Moving the call outside the loop would collapse the batch boundaries and defeat the scheduler's concurrency constraint.

2.6. Related Work

2.6.1. MoE communication and load balancing

Most MoE literature treats balance at training time, routing time, or expert placement time. Load-balancing losses and bias updates aim to prevent router collapse [6, 25]. Distributed inference adds a systems bottleneck: expert parallelism interleaves computation with all-to-all communication. Systems including FasterMoE, Tutel, and MegaBlocks optimize dynamic routing, parallel layouts, or sparse expert computation [7, 8, 10]. FlexMoE goes further by changing expert placement in response to observed load [14]. These systems establish that routing skew must be handled at runtime, but they do not schedule vLLM's background EPLB weight migration against concurrent inference. Lina shows that skewed expert popularity creates imbalanced transfer sizes and bandwidth demand, and reports substantial P95 inference latency improvements from communication-aware scheduling [13]. COMET independently reports that inter-device communication can account for up to 47% of a MoE layer's execution time in the evaluated models and frameworks [29]. These measurements motivate treating network interference as a first-class serving concern.

They do not, however, imply a universal fixed link-utilization ratio.

This work begins after EPLB has already chosen a valid new placement. It does not change router probabilities, model outputs, or the placement objective. Instead, it schedules the physical copying needed to realize that placement. A placement algorithm decides *what* state should move; the migration scheduler decides *when* independent transfers should share network resources. The two decisions are complementary.

2.6.2. LLM serving interference and runtime movement

Iteration-level batching in Orca and PagedAttention in vLLM show that online request scheduling is essential to serving efficiency [11, 28]. More recent systems isolate different forms of interference: DistServe separates prefill from decode, Sarathi-Serve uses chunked prefills to avoid decode stalls, and Llumnix migrates live request state between instances to improve load balance and tail latency [1, 21, 30]. The present work addresses a different object and time scale. It does not move requests or KV caches and does not change request batching; it schedules expert-weight copies that arise only when EPLB installs a new placement.

2.6.3. Collective-flow scheduling

A set of expert migrations resembles a coflow: several point-to-point flows share a common completion or interference objective. Varys models coflow completion through the most heavily loaded ingress or egress endpoint and the remaining measured bandwidth at that endpoint [3]. This bottleneck view is closely related to the maximum normalized link-load term in the ILP developed in chapter 3. The present model is intentionally simpler: it uses fixed routes and a normalized foreground-traffic parameter, and it optimizes a serving-interference surrogate rather than coflow completion time.

2.6.4. Graph scheduling and expert migration

A migration layer can be represented by a graph whose vertices are ranks and whose directed edges are remote flows. If each rank may participate in at most one flow in a batch, edges sharing an endpoint cannot receive the same batch identifier. This is an edge-coloring constraint [26]. Transfer direction and expert identity remain necessary for execution, but direction does not change whether two flows contend at an endpoint. Each generated batch is a matching in this rank graph.

Exact edge coloring alone is not the complete systems objective: topology, transfer sizes,

replica choices, and foreground traffic also matter. The deployed first-fit rule is therefore a conservative, topology-independent heuristic rather than a claim of general optimality. It is inspired by the migration grouping used in OmniInfer's placement manager [17]. The contribution of this thesis is the explicit interference formulation and the integration of this rule into vLLM's asynchronous EPLB path, including deterministic flow construction, NIXL lifecycle handling, cycle-wide schedule precomputation, correctness tests, and end-to-end serving evaluation.

2.6.5. Positioning of this thesis

Prior MoE systems primarily optimize token dispatch, expert computation, or placement, while coflow systems optimize transfer completion and LLM-serving systems schedule requests or KV-cache movement. This thesis begins after an EPLB policy has selected a placement and asks how to realize that decision without producing avoidable NIC bursts during inference. Its distinct contribution is the combination of an interference-oriented formulation, a dependency-free rank-pair batching rule, communicator-aware execution, and an upstream vLLM implementation evaluated with both serving and NIC measurements.

3 | Problem Statement and Formulation

3.1. Migration instructions and flows

Consider one MoE layer before and after a rebalance. Let R be the set of EP ranks and E the set of logical experts. The old and new placement arrays contain the expert identifier stored in every physical slot. Comparing the two arrays determines which rank gains each expert. If a destination already owns a copy, only a local rearrangement is needed; otherwise, one existing source copy must supply the new replica.

A raw migration instruction is a tuple (s, d, e) containing source rank s, destination rank d, and logical expert e. Multiple experts may have the same ordered rank pair. Since such transfers have identical endpoint constraints, the scheduler coalesces them into a flow

$$p = (s_p, d_p, E_p), \tag{3.1}$$

where E_p is the ordered set of expert identifiers assigned to the pair. Let $w_p = |E_p|$ be the flow size in expert units. Coalescing reduces scheduler work and prevents identical rank pairs from being artificially separated across batches.

When an expert has several old replicas and several new destinations, the source choice is also a scheduling decision in principle. The production implementation preserves the existing vLLM assignment: destinations are distributed as evenly as possible over source replicas in deterministic rank order. Migration batching changes ordering, not placement semantics or replica ownership.

3.2. Problem inputs and output

The optimization instance has five input groups: the coalesced migration flows P and their sizes w_p; the candidate batch set B; the routed-link set $\mathcal{L}$ with capacities C_e; the

fixed route fractions $f_{p,e}$; and the foreground parameters T_0 and ρ. The optional diagnostic additionally uses a fixed per-batch overhead δ. These values describe what must move, where it can travel, and how much network headroom remains for migration.

The required output is a batch assignment for every flow. Equivalently, the solver returns the binary variables $x_{p,b}$ and y_b, from which the ordered list of active batches is reconstructed. A valid output schedules every input flow exactly once. The primary objective is the smallest worst-case modeled inference time; among schedules with that value, the secondary objective is the smallest number of active batches.

3.3. Network model

The offline model supports an explicit network graph and a compact two-tier topology. In the latter, GPUs connect to server endpoints through local links; server endpoints share a slower inter-server network. Let $\mathcal{L}$ be the set of routed links and $C_e > 0$ the capacity of link $e \in \mathcal{L}$. Fixed shortest-path ECMP determines $f_{p,e}$, the fraction of flow p routed over e. In the illustrative instance there is a single path for each flow, so $f_{p,e}$ is binary.

The model measures flow size in expert units rather than bytes. It therefore assumes equal expert payloads within the layer. This matches the evaluated model architecture but would need extension for heterogeneous or quantized experts. Following the bottleneck and residual-bandwidth view used in coflow scheduling [3], it represents the bandwidth left by foreground inference as $C_e(1-\rho)$, where $\rho \in [0, 1)$ is the fraction of link capacity occupied by inference. Varys estimates residual bandwidth from measurements; the illustrative ILP instead uses one normalized ρ for all links. It is therefore a scenario parameter, not a deployment-independent empirical constant.

3.4. Constrained integer linear program

Let P be the set of coalesced flows and $B = \{0, \ldots, |P| - 1\}$ a safe upper bound on candidate batches. The binary assignment variable is

$$x_{p,b} = \begin{cases} 1 & \text{if flow } p \text{ is assigned to batch } b, \\ 0 & \text{otherwise.} \end{cases} \tag{3.2}$$

Binary variable y_b indicates whether batch b is active, and T_{infer} is a continuous variable for the worst modeled inference time. Every flow is assigned exactly once:

$$\sum_{b \in B} x_{p,b} = 1 \qquad \forall p \in P. \tag{3.3}$$

Assignments activate their batch, and symmetry is reduced by requiring active batches to form a prefix:

$$x_{p,b} \leq y_b, \tag{3.4}$$

$$y_b \geq y_{b+1}. \tag{3.5}$$

The migration load on routed link e in batch b is

$$L_{e,b} = \sum_{p \in P} w_p f_{p,e} x_{p,b}. \tag{3.6}$$

Let T_0 be inference time without migration. The model assumes slowdown is linear in migration load relative to residual link capacity:

$$T_{\text{infer}} \geq T_0 \left(1 + \frac{L_{e,b}}{C_e(1-\rho)}\right) \qquad \forall e \in \mathcal{L},\ \forall b \in B. \tag{3.7}$$

The maximum-over-links structure follows the standard bottleneck observation that a collection of parallel network flows is limited by its most heavily loaded endpoint or link [3]. The primary optimization is

$$T^*_{\text{infer}} = \min T_{\text{infer}}. \tag{3.8}$$

Under this model, the first solve minimizes the maximum normalized link load over all link–batch pairs. After obtaining T^*_{infer}, the solver runs a second optimization:

$$B^* = \min \sum_{b \in B} y_b \quad \text{subject to} \quad T_{\text{infer}} \leq T^*_{\text{infer}} + \epsilon. \tag{3.9}$$

Here, ϵ is only a numerical tolerance. The second stage removes unnecessary batches but cannot exchange a meaningful increase in inference time for fewer batches. This lexicographic formulation avoids an arbitrary weighted sum between foreground latency and batch overhead. The implementation therefore performs two actual mixed-integer solves through SciPy's interface to HiGHS [9, 22].

3.5. Migration-time diagnostic

The ILP deliberately optimizes modeled inference time, not migration time. To make the trade-off visible, it reports a diagnostic. The duration of batch b is approximated by its bottleneck-link service time:

$$T_b = \max_{e \in \mathcal{L}} \frac{L_{e,b}}{C_e}. \tag{3.10}$$

Given a per-batch launch or synchronization overhead δ, total migration time is estimated as

$$T_{\text{migration}} = \sum_b T_b + \delta \sum_b y_b. \tag{3.11}$$

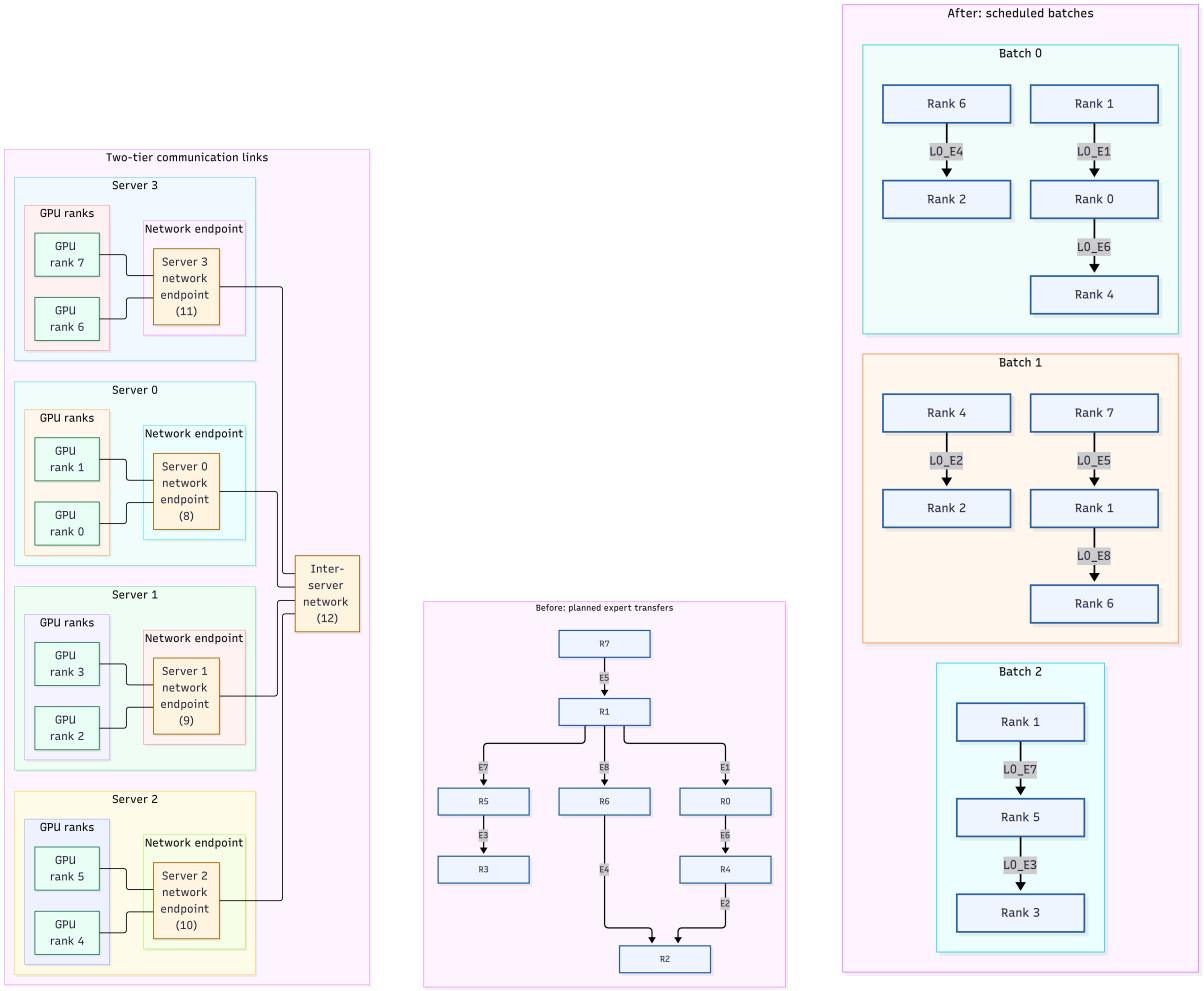
Splitting one busy batch into several lighter batches can reduce the maximum in eq. (3.7) while increasing the sum in eq. (3.11). This is not a contradiction. The two quantities answer different questions: the first represents foreground interference; the second represents how long the background reconfiguration remains unfinished.

3.6. Illustrative hotspot instance

The example contains eight GPU ranks, two GPUs per server, local-link capacity 18, network-link capacity 2, and $\rho = 0.6$, leaving 40% normalized link headroom for migration. Eight migrations create a shared-network hotspot. The numeric units and the 40% headroom are a deliberately stressed illustrative scenario, not measurements from the experimental cluster. Because the same ρ multiplies every candidate link–batch pair, changing it uniformly rescales the reported value of T_{infer} but does not change which batch assignment minimizes the objective. A production predictor should replace it with per-link residual bandwidth measured at runtime.

Figure 3.1 shows the routed topology, the unbatched hotspot, and the schedule returned by the two-stage ILP. Arrow labels identify logical experts, while the batch labels in the rightmost panel show which transfers execute together.

As shown in table 3.1, the optimum reduces peak link utilization by a factor of three and modeled inference time by more than half. The secondary objective selects three batches among schedules with the same primary optimum; the estimated migration time rises only because the diagnostic includes a fixed cost for every batch. This result supplied



(a) Two-tier routed-link topology.

(b) All migrations issued together.

(c) ILP schedule split into batches.

Figure 3.1: Illustrative hotspot topology and migration schedules before and after contention-aware batching. The unbatched diagram remains a directed rank graph; only its layout is vertical.

Table 3.1: Result of the illustrative ILP instance. Values are normalized.

Metric	Unbatched	ILP optimum
Modeled inference time	4.75	2.25
Estimated migration time	1.55	1.65
Peak link utilization	1.50	0.50
Number of batches	1	3

the design insight for the online scheduler: spread contending flows while preserving useful parallelism. The production scheduler realizes that insight with a simpler, conservative rank-disjoint rule; that rule is not imposed on this ILP.

3.7. Why the ILP is not used online

The optimizer is intentionally separated from the serving path. It is not an "optimal implementation" of EPLB: it is an exact offline reference for one restricted mathematical formulation. First, an online solver would add SciPy/HiGHS dependencies and variable solve latency to a periodic control operation. Second, the latency model in eq. (3.7) is an abstraction. Real interference depends on transport protocols, queueing, CPU progress, message sizes, and simultaneous inference collectives. An exact optimum of an approximate model is not necessarily a better production choice than a conservative rule.

Third, all ranks already possess the placement maps needed to construct the same deterministic schedule. A lightweight local algorithm avoids a new coordination service. The ILP remains valuable for explaining the objective, checking small instances, and demonstrating the value of contention-aware batching. The next chapter describes the conservative rule used by the deployed scheduler.

4 | Methodology and System Design

4.1. Design requirements

The production design follows six requirements derived from the vLLM runtime.

1. **Semantic preservation.** The same new placement must be reached as in the unbatched path. Scheduling may change transfer order, but not the selected logical experts, destination slots, or replica assignment.
2. **No endpoint contention inside a batch.** Each rank participates in at most one coalesced directed flow, whether as sender or receiver. Independent rank pairs should remain concurrent.
3. **Determinism.** Every rank builds the same batch structure from replicated placement arrays without exchanging a separate schedule.
4. **Low control-plane cost.** The algorithm runs repeatedly during serving. It must avoid an optimizer and minimize Python object construction and repeated function-entry overhead.
5. **Communicator compatibility.** Batch boundaries must be visible to the communication backend, and NIXL's transfer context lifecycle must be respected.
6. **Safe rollout.** The new behavior is disabled by default and is reachable only in asynchronous EPLB. Existing synchronous users retain the established migration path.

4.2. From placements to remote flows

For each rank, the scheduler converts its old and new local slot arrays into sets of logical expert identifiers. The difference $H_r^{\text{new}} \setminus H_r^{\text{old}}$ identifies experts that rank r must receive

remotely. An expert already present locally does not generate a remote instruction even when its physical row changes.

The algorithm then indexes old source ranks only for expert identifiers that are needed somewhere. This filtering avoids building a global inverse index for experts irrelevant to the current migration. If several source replicas exist, the ordered destinations are divided evenly among them using quotient and remainder assignment. This reproduces the legacy source-balancing logic.

Finally, expert identifiers are grouped in a dictionary keyed by (s, d). One `MigrationFlow` contains the source rank, destination rank, and immutable tuple of experts for that ordered pair. Direction is retained for send/receive execution, although the contention test treats both endpoints symmetrically.

4.3. Greedy rank-disjoint grouping

The communication graph has rank vertices and one coalesced edge for each directed pair. A batch is a matching. The scheduler processes flows in deterministic insertion order and places each in the first existing batch whose endpoint set is disjoint. If no batch is compatible, it creates a new one.

Instead of maintaining a Python set for every batch, the implementation uses an integer bit mask. For a flow (s, d),

$$m_{s,d} = (1 \ll s) \ \vee \ (1 \ll d). \tag{4.1}$$

If u_b is the mask of ranks already used in batch b, the flow fits exactly when $m_{s,d} \wedge u_b = 0$. On insertion, u_b becomes $u_b \vee m_{s,d}$. This is constant-time endpoint testing for practical EP group sizes.

Algorithm 4.1 gives the complete grouping procedure. The input order is deterministic on every rank, and the endpoint bit mask is both the admission test and the state updated after a flow is inserted.

The greedy algorithm does not promise the minimum number of colors for every graph. That is acceptable because batch count is secondary to a simple and conservative concurrency cap. On four ranks, a batch contains at most two independent directed flows. The rank-disjoint rule intentionally leaves headroom rather than trying to saturate the NIC with migrations. Reciprocal flows such as $r_0 \to r_1$ and $r_1 \to r_0$ are distinct directed flows

```
Input: coalesced flows P in deterministic order
batches = []; used = []
for flow p = (src, dst, expert_ids) in P:
    mask = (1 << src) | (1 << dst)
    for batch_index in range(len(batches)):
        if mask & used[batch_index] == 0:
            batches[batch_index].append(p)
            used[batch_index] |= mask
            break
    else:
        batches.append([p])
        used.append(mask)
return batches
```

Algorithm 4.1: Deterministic first-fit migration batching

and therefore occupy separate batches, even though they involve the same rank pair.

4.4. Correctness invariants

The scheduler and execution path maintain three key invariants.

Completeness. Each remote gain appears in exactly one coalesced pair and each pair is inserted in exactly one batch. Hence every derived migration is scheduled once.

Conflict freedom. Suppose two flows in the same batch share rank r. Both masks contain bit r. When the second flow is considered, its mask intersects the batch mask, so the algorithm rejects that batch. This contradicts co-residence; therefore flows in a batch have disjoint endpoints.

Local-copy preservation. Experts already available on a destination rank are excluded before remote flow construction. The existing local copy and receive-primary metadata still control physical-row updates. Batching modifies only the remote communication phase.

Unit tests cover deterministic grouping, rank-pair coalescing, random instruction completeness, exclusion of local copies, replica balancing, multi-layer equivalence, and execution with a mock communicator. The upstream change added the test file to vLLM's expert-parallel CI area.

4.5. Complexity

Let S be the number of physical slots across all ranks in one layer, M the number of remote expert migrations, F the number of distinct directed rank pairs after coalescing, and K the number of produced batches. Placement scanning is $O(S)$. Building and grouping the migrations is $O(M)$ expected time with hash tables. First-fit placement is $O(FK)$ in the worst case and $O(F)$ storage, in addition to the placement indices.

For a fixed small number of ranks, K is bounded and the practical cost is close to linear in placement size plus migration count. More importantly, the implementation invokes the outer scheduler once for all layers in a rebalance cycle. If the model contains L MoE layers, it still constructs L independent schedules, but avoids L separate profiled entry points and repeated surrounding Python setup.

4.6. Cycle-wide precomputation, layer-wise transfer

An earlier design called the scheduler immediately before each layer transfer. The final design receives two-dimensional old and new placement arrays and computes a list of schedules—one per layer—inside one cycle-wide call. This does *not* combine expert payloads from different layers. The asynchronous worker later iterates over layers and passes only the corresponding schedule to the transfer function.

The distinction is important. Weight transfers and mapping activation preserve the existing layer lifecycle, limiting memory and synchronization changes. Only CPU schedule construction is aggregated. Qwen3-30B-A3B has 48 layers, so one profiling range represents schedule construction for all 48 layers. The worker waits for the new placement maps before this step; adding a few milliseconds of CPU work once per rebalance cycle is negligible relative to the measured cycle interval, but this claim is verified rather than assumed in section 5.2.3.

4.7. Batched execution

For a layer, the executor first creates maps from logical expert identifier to the old and new local physical row. It then processes the ordered list of batches. For every batch it:

1. restores the communicator's transfer context for the current old placement and layer;
2. posts sends for flows whose source is the local EP rank;
3. posts receives into expert buffers for flows whose destination is the local EP rank;

and

4. calls `communicator.execute()` to preserve the batch boundary.

For NIXL, restoring context per batch is required because execution drains the queued operations and clears the transfer context. Calling `execute()` once after all batches would submit the operations together, erasing the very boundaries that enforce rank-disjoint concurrency. Conversely, an uninvolved rank can have no send or receive in a given batch; the backend and existing layer coordination govern progress.

4.8. Configuration and failure containment

The change adds `enable_migration_batching` to `EPLBConfig`, with default `false`. The effective property is

$$\texttt{migration_batching_enabled} = \texttt{use_async} \wedge \texttt{enable_migration_batching}. \quad (4.2)$$

Thus, setting the new flag cannot alter synchronous EPLB. This is a deliberate engineering boundary. In synchronous mode, serving is stopped and serial batches can only delay its resumption. In asynchronous mode, the operator can enable batching when network interference is a concern. Keeping the feature opt-in also acknowledges that the current evidence comes from one cluster and that the optimal concurrency limit may vary with topology.

When disabled, the original path constructs local metadata, posts all remote sends and receives, and invokes the communicator once. The feature does not change model routing, balancing policy, or numerical output. This small impact surface was important for upstream review and made rollback a configuration change rather than a code revert.

4.9. Upstream integration

The final patch added a scheduler module, unit tests, one configuration field, async-worker wiring, and a batched branch in the migration executor. It did not ship the offline ILP, datasets, experimental scripts, or thesis artifacts as runtime dependencies. The contribution was merged into vLLM on 3 October 2026 as pull request #52641 [2]. The separation between research artifacts and production files keeps the upstream change reviewable and avoids coupling online serving to the experimental environment.

5 | Experimental Evaluation

5.1. Methodology

5.1.1. Evaluation goals

The experiments answer three operational questions:

1. Does enabling migration batching improve user-visible serving metrics under asynchronous EPLB?
2. Does it reduce high-percentile network traffic, the mechanism the scheduler is designed to affect?
3. Is the CPU scheduling overhead small relative to the observed E2E time saved?

These questions lead to three directional expectations rather than formal statistical hypotheses. First, batching should lower the high tail of observed NIC traffic because fewer migration flows contend at one rank. Second, if that reduction protects foreground communication, output throughput should increase and TTFT, TPOT, and E2E latency should decrease. Third, cycle-wide schedule construction should consume only a small fraction of the serving time saved. The single A/B pair can check whether the observations have these directions, but it cannot establish significance or estimate run-to-run variance.

Two separate runs are used for two separate purposes. A clean A/B serving run compares batching off and on without profiler instrumentation. A profiling run repeats the batching-on condition with NVTX-only Nsight Systems capture. This separation prevents profiler overhead from contaminating the reported serving comparison.

5.1.2. Hardware and software environment

Table 5.1 records the hardware, software, network, model, and profiling tools used for the final A/B comparison.

The four GPU processes serve as both TP and EP ranks. One GPU per host makes

Table 5.1: Experimental environment.

Component	Configuration
Nodes	Four Ubuntu 22.04.5 servers
Accelerators	One NVIDIA Quadro RTX 6000, 24 GB, per node
Network	10 Gb/s Ethernet, interface `eno2`, no RDMA
Model	Qwen/Qwen3-30B-A3B-Instruct-2507, FP16
Parallelism	Tensor parallel size 4; expert parallel enabled
Runtime	Experimental vLLM build; Ray 2.56.1
Migration backend	NIXL 1.3.2 over UCX/TCP
Execution	Eager mode; prefix caching enabled
Profiling	NVIDIA Nsight Systems 2024.5.1, NVTX-only

Table 5.2: EPLB parameters common to the A/B experiment.

Parameter	Value
`window_size`	50
`step_interval`	50
`num_redundant_experts`	32
`log_balancedness`	false
`use_async`	true
`communicator`	`nixl`
`enable_migration_batching`	false / true

each inter-rank expert transfer an inter-node transfer, avoiding ambiguity between local NVLink/PCIe movement and Ethernet traffic. The hardware is not a modern production cluster; its relatively constrained network is useful for observing contention but limits external validity.

NCCL and Gloo are bound to `eno2`, InfiniBand is disabled, and UCX is also bound to the same device. NIXL is selected explicitly rather than relying on automatic backend selection. Both A/B conditions use the same software revision and network configuration.

5.1.3. Model and EPLB configuration

The evaluated model contains 48 layers and 128 experts, selecting eight experts per token [18]. Server maximum sequence length is 4096 and GPU memory utilization is set to 0.9. EPLB uses the configuration in table 5.2.

The short observation and step intervals deliberately cause recurring rebalances during the finite benchmark. This is a stress-oriented configuration for studying the migration path rather than a recommended production default. The 32 redundant experts create

physical flexibility for EPLB to react to recent routing demand.

5.1.4. Workload

Each condition starts a fresh server and executes an eight-request warm-up. The measured workload uses `vllm bench serve` with 500 successful requests and maximum concurrency 32. Every request has a 300-token random prefix, 200 additional input tokens, and 300 requested output tokens. The random seed is zero, temperature is zero, and end-of-sequence tokens are ignored so all requests produce the configured output length. Prefix caching remains enabled, matching the intended online configuration rather than disabling cache to manufacture compute load.

The request set is synthetic. It controls sequence lengths and makes the A/B conditions comparable, but random tokens are not representative of semantic production traffic. The experiment studies system interference under frequent migration, not application-level output quality. Because placement batching does not modify model computation or routing decisions, no accuracy evaluation is required for the feature itself.

5.1.5. Baseline and proposed condition

The baseline disables migration batching and uses vLLM's legacy migration execution. For each layer, it posts every remote send and receive before one communicator execution, so migration has no scheduler-created batch boundaries. The proposed condition enables only the `enable_migration_batching` flag. It derives the same migrations and reaches the same expert placement, but groups coalesced rank-pair flows with the algorithm in algorithm 4.1 and executes the resulting batches sequentially. This baseline isolates the contribution under study; it is not a different placement policy, model, workload, or communication backend.

5.1.6. A/B procedure and controls

The independent variable is `enable_migration_batching`. All other documented parameters are held constant. The off case ran first and the on case second, each with a fresh server. Both completed 500 of 500 requests with zero failures. The benchmark client records aggregate duration and request-level latency distributions.

The experiment is a single repetition. It therefore cannot quantify run-to-run variance, confidence intervals, or statistical significance. Percent changes are descriptive comparisons of these archived runs. A stronger study would randomize condition order and repeat each

condition enough times to estimate variance. This limitation is stated explicitly rather than obscured by the large number of requests, because requests within one long run are not independent replications of server startup and cluster state.

5.1.7. Serving metrics

For a metric where larger is better, such as throughput, improvement is

$$I_{\uparrow} = \frac{V_{\text{on}} - V_{\text{off}}}{V_{\text{off}}} \times 100\%. \tag{5.1}$$

For a latency metric where smaller is better,

$$I_{\downarrow} = \frac{V_{\text{off}} - V_{\text{on}}}{V_{\text{off}}} \times 100\%. \tag{5.2}$$

The thesis reports output tokens per second and P50/P99 TTFT, TPOT, and E2E latency. Request throughput is retained in the raw JSON and the supplementary table. Because all 500 requests produce exactly 300 output tokens, request and output throughput necessarily have the same relative change; the main text uses output throughput to avoid presenting this one result twice. P99 values are computed by the benchmark client from the 500 request samples.

5.1.8. NIC measurement

NIC traffic is not produced by `vllm bench serve`. A separate sampler on the head node reads the cumulative receive and transmit byte counters under `/sys/class/net/eno2/statistics/` once per second. For adjacent samples at times t_{i-1} and t_i, aggregate traffic is

$$B_i = \frac{(RX_i - RX_{i-1}) + (TX_i - TX_{i-1})}{t_i - t_{i-1}}. \tag{5.3}$$

Values are converted to MB/s and the P99 is computed over the formal benchmark window. Summing RX and TX describes total head-node interface pressure; it is not physical link utilization and it cannot attribute bytes to EPLB versus inference. The A/B comparison and time-series shape provide supporting evidence, not packet-level causal attribution.

5.1.9. Scheduler profiling

The profiling run uses the same model, EPLB settings, workload lengths, concurrency, and 500 prompts as the batching-on E2E condition. The only change is that Ray processes are launched in NVTX-only Nsight Systems sessions with CPU sampling and context-switch tracing disabled. NVTX ranges are designed to mark application phases with low capture volume [16].

The range `eplb: schedule migration batches` encloses the call that builds schedules for all 48 layers. Each rank writes its own `rank*.nsys-rep`. Explicit formal-start and formal-end timestamps exclude model loading and warm-up. The reports were analyzed with Nsight Systems 2024.5.1. For each rank, the number of calls, P50 and P99 elapsed duration per call, and cumulative duration are extracted.

The explicit CUDA NVTX range was temporary measurement instrumentation added only to make the asynchronous EPLB worker visible to Nsight Systems. It was used in the profiling run, was absent from the clean E2E A/B run, and was not included in the final production patch. The scheduler call and placement inputs were otherwise unchanged.

Ranks construct their identical schedules concurrently on different host CPUs. Their costs must therefore not be summed to estimate critical-path overhead. A conservative comparison uses the largest cumulative rank cost, $C_{\max}$, and clean A/B time saved,

$$S = T_{\text{off}} - T_{\text{on}}. \tag{5.4}$$

The cost-to-saved ratio is

$$R = \frac{C_{\max}/1000}{S} \times 100\%. \tag{5.5}$$

This ratio does not claim that all scheduling time lies on the serving critical path. It intentionally compares a directly measured conservative CPU elapsed cost with the directly measured wall-clock difference.

5.1.10. Data provenance and reproducibility

The authoritative E2E source directory is `results/serving_nixl_20260911_async_random500`. It contains the unmodified off/on `bench.json`, server and client logs, EPLB configuration, one-second `nic.tsv`, warm-up output, plotting source, and summary CSV files. The authoritative profiling directory is the same name with suffix `_profile`; it con-

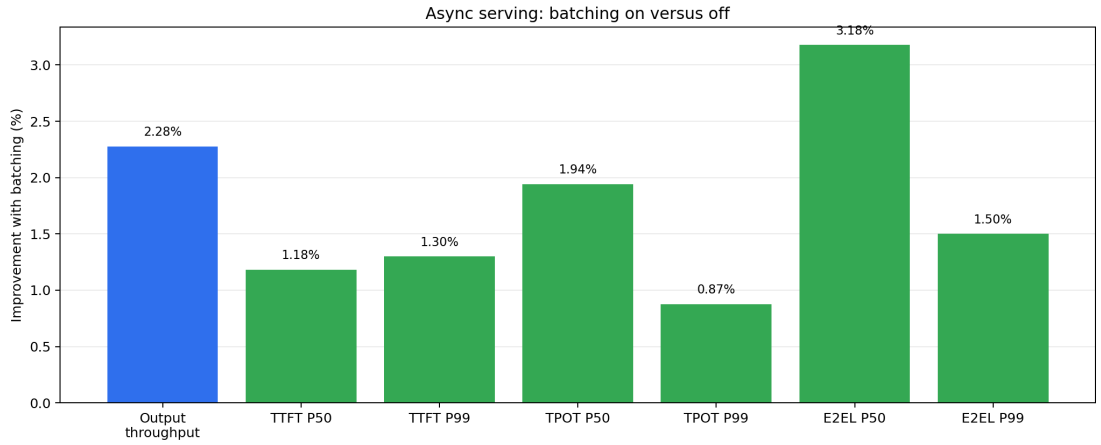


Figure 5.1: Relative improvement with batching enabled in the clean A/B run. Positive values indicate higher throughput or lower latency.

tains four Nsight reports, formal-window timestamps, benchmark output, exact commands, and `profile_summary.csv`.

Older directories represent intermediate designs or exploratory workloads and are excluded from all final tables. Reproduction commands are transcribed in appendix A. This explicit provenance prevents accidental selection of a favorable intermediate run.

5.2. Results and Discussion

5.2.1. End-to-end serving results

Table 5.3 reports the clean A/B measurement. Both conditions completed all 500 requests. Enabling batching shortened total benchmark duration from 4756.098 s to 4650.166 s, a difference of 105.932 s. Output throughput increased from 31.54 to 32.26 tokens/s, or 2.28%.

All six reported latency percentiles moved in the favorable direction. Median/P99 TTFT decreased by 1.18%/1.30%; median/P99 TPOT decreased by 1.94%/0.87%; and median/P99 E2E latency decreased by 3.18%/1.50%. The larger improvement in median E2E than in an individual TTFT or TPOT percentile should not be interpreted as an algebraic sum of percentile changes: each percentile may correspond to a different request, and E2E accumulates many decode intervals.

Figure 5.1 visualizes the same table as signed relative improvements, using positive values for both higher throughput and lower latency.

The direction is consistent with the hypothesis that migration bursts interfere with inference. TTFT improves despite batching being a background migration optimization because the benchmark mixes prefill and decode across concurrent requests. TPOT improves because decode continues throughout asynchronous migration. The result does not isolate which inference communication operation benefited; it measures the user-visible aggregate effect.

5.2.2. Network traffic

The head-node P99 of aggregate RX+TX traffic decreased from 599.81 MB/s to 538.40 MB/s, a reduction of 10.24%. This is the largest relative change among the reported measurements and directly concerns the mechanism targeted by the scheduler: smoothing extreme network demand rather than reducing total expert bytes.

Figure 5.2 shows that traffic is bursty in both configurations. This is expected: inference itself communicates and EPLB continues to migrate whole expert tensors. The scheduler does not impose a byte-rate limiter. It limits per-rank flow concurrency, spreading conflicting migration flows across rounds. The lower P99 supports the contention explanation, but the two time series are not temporally aligned executions of identical low-level events. They are separate A/B runs with the same client workload.

The measurement also has a scope limitation. Only the head node was sampled, so the result does not prove that every worker's P99 decreased. Because the scheduler enforces the same rank-degree constraint globally, lower bursts are plausible on other ranks, but per-node counters are necessary to verify that claim.

5.2.3. Scheduler profiling

All four ranks recorded 34 calls to the cycle-wide scheduling range during the formal profiling window. Since one call prepares all 48 layer schedules, the trace represents 34 rebalance-cycle scheduling events per rank, not 34×48 independent top-level calls. The per-rank distributions and cumulative durations are reported in table 5.4.

Median elapsed time is between 7.01 and 7.82 ms per rank per rebalance cycle. Dividing by 48 gives roughly 0.146–0.163 ms per layer as an intuition, but the implementation and profiler measure one aggregated call, so this quotient is not presented as an independently observed per-layer latency. P99 variability is larger, consistent with host scheduling and interference in the background thread.

Rank 0 has the largest cumulative cost, 302.081 ms. Ranks execute on separate hosts

Table 5.3: End-to-end serving results. Lower is better for all latency columns; higher is better for throughput. Each condition is one run.

Batching	Duration (s)	Output tok/s	TTFT P50	TTFT P99	TPOT P50	TPOT P99	E2E P50 / P99
Off	4756.10	31.54	49180.71	75740.25	828.53	1007.82	303462.69 / 322184.88
On	4650.17	32.26	48599.04	74756.17	812.44	999.01	293821.76 / 317351.61

Latency values are milliseconds.

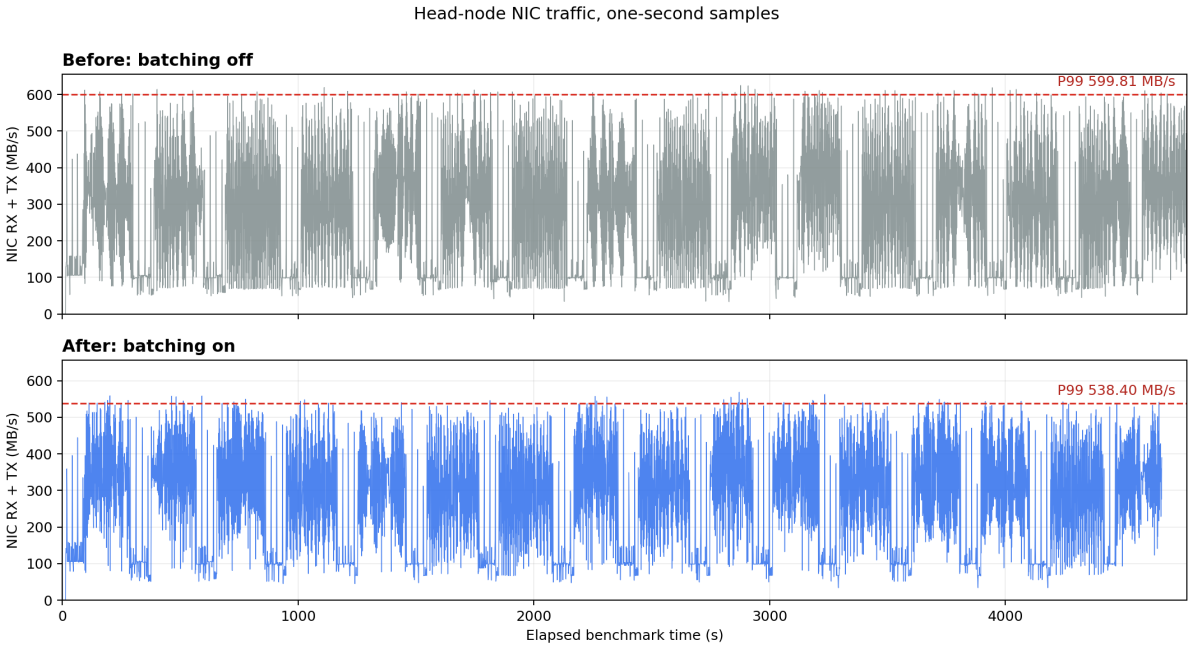


Figure 5.2: Head-node aggregate NIC traffic sampled once per second during the two independent runs. Batching reduces the high tail of observed traffic.

Table 5.4: NVTX elapsed time for cycle-wide scheduler calls.

Rank	Calls	P50/call (ms)	P99/call (ms)	Cumulative (ms)
0	34	7.819	17.536	302.081
1	34	7.090	12.357	253.152
2	34	7.035	16.104	255.239
3	34	7.005	8.103	241.190

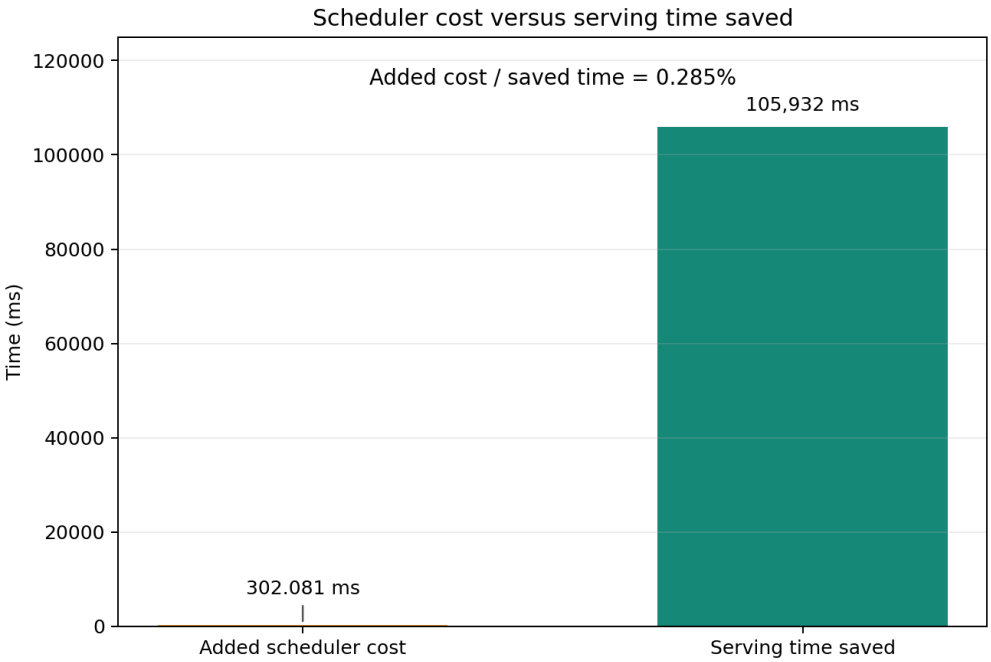


Figure 5.3: Largest measured cumulative scheduler time versus serving time saved. The linear axis intentionally preserves the scale difference.

and schedule concurrently, so adding the four cumulative values would overstate wall-clock overhead. Applying eq. (5.5) to the clean A/B saving gives

$$R = \frac{0.302081}{4756.098161 - 4650.165779} \times 100\% = 0.285\%. \tag{5.6}$$

Figure 5.3 keeps both quantities on a linear scale: the small orange bar is the largest measured cumulative scheduling cost on one rank, and the green bar is the independent A/B serving-time difference. This comparison shows that the measured scheduler cost is small relative to the observed completion-time difference. It is not a causal decomposition of the saving: the profiler run is separate, and the 105.932 s difference includes all downstream effects. It nevertheless answers the maintainer-facing systems question of whether the additional Python scheduling work is of the same order as the apparent benefit. It is not.

5.2.4. Answering the research questions

RQ1: representation. The directed migration-flow graph captures endpoint contention cleanly, while the topology-aware ILP adds routed-link capacity for offline reasoning. The ILP example confirms that minimizing foreground contention can intentionally increase migration completion time.

RQ2: online scheduling. Coalescing by directed rank pair followed by first-fit edge coloring provides a deterministic and dependency-free rule. The bit-mask representation makes the central constraint explicit and inexpensive. It is not guaranteed to minimize batch count, but exact color optimality is not required for safe throttling.

RQ3: semantic integration. Tests show completeness, conflict freedom, local-copy exclusion, replica balancing, deterministic output, and correct mock-communicator execution. The implementation restores NIXL context and executes each batch separately, precomputes all layer schedules once per cycle, and leaves synchronous and disabled paths unchanged.

RQ4: serving impact. In the evaluated cluster, every reported throughput or latency metric improved, and NIC P99 decreased by 10.24%. Scheduler profiling found a conservative cost-to-saved ratio of 0.285%. These results are consistent with reduced migration interference and justify the implementation for this environment.

5.2.5. Discussion

The results illustrate why migration duration alone is the wrong optimization target for asynchronous EPLB. A scheduler can add communication rounds yet improve total serving completion time, because the foreground model continues to execute during those rounds. The relevant resource is not only bytes moved but how aggressively those bytes are injected while latency-sensitive traffic is active.

The rank-disjoint-flow constraint is deliberately conservative. On n ranks, at most $\lfloor n/2 \rfloor$ independent directed flows appear in a batch. A topology-aware scheduler might safely allow greater rank participation when ranks are co-located, use separate NICs, or have available bandwidth. The fixed rule trades peak migration speed for predictable behavior and portability without topology discovery.

Cycle-wide precomputation addresses a different concern: CPU overhead. It does not make network batches coarser and does not transfer 48 layers together. It reduces top-level

scheduling invocations while preserving layer-wise migration. This separation between scheduling granularity and transfer granularity is a key engineering result of the work.

5.2.6. Threats to validity

Internal validity

The off run preceded the on run, so temporal drift, thermal state, background load, or network state may contribute to the difference. Fresh servers and an identical warm-up reduce but do not eliminate these effects. Only one run per condition was retained; repeated interleaved runs would provide stronger evidence. The client uses the same seed, yet stochastic system scheduling means request timing is not identical.

NIC counters include all interface traffic. They cannot distinguish NIXL migration bytes from Ray control traffic, model communication, or unrelated traffic. A packet or transport-level trace and counters on every node would strengthen causal attribution. Still, the lower NIC P99 and favorable serving metrics are mutually consistent.

Construct validity

The ILP slowdown equation is a deliberately linear surrogate. Queueing and protocol effects are nonlinear, and flow weights count experts rather than bytes or measured transfer time. The ILP result should therefore be read as a design demonstration. The E2E benchmark, not the ILP objective value, is the evidence for real serving behavior.

The random-token workload fixes lengths and activates frequent EPLB, but it is not a production request distribution. Ignoring EOS makes output work deterministic but removes natural variation. Prefix caching is enabled, which improves realism relative to disabling it, although random prefixes offer limited sharing.

External validity

The cluster has four older GPUs and 10 Gb/s Ethernet without RDMA. Production systems may use NVLink within nodes and 100–400 Gb/s RDMA between nodes. Higher bandwidth can reduce contention, whereas larger models and more ranks can increase the number and size of migrations. Results may therefore be smaller or larger in another environment.

Only Qwen3-30B-A3B-Instruct-2507 and NIXL 1.3.2 were used in the final study. The code supports the vLLM communicator abstraction, but backend-specific progress and synchronization behavior can change batch overlap. Separate evaluation is needed for

Gloo and future NIXL versions.

Conclusion validity

The results are effect estimates from a single A/B pair, not confidence intervals. The thesis consequently avoids terms such as “statistically significant” and does not generalize the exact percentages. The strongest supported conclusion is conditional: under this documented configuration, batching coincided with lower NIC tail traffic and improvements in all reported serving metrics at a small measured scheduling cost.

6 | Conclusions

This thesis investigated a narrow systems problem created by dynamic expert placement: how to migrate expert weights without allowing background network traffic to interfere unnecessarily with ongoing inference. The key conceptual step was to distinguish migration completion time from foreground serving quality. In synchronous EPLB, finishing the migration quickly is appropriate because inference is paused. In asynchronous EPLB, inference and migration share resources, so lower peak contention can be worth additional transfer rounds.

An offline ILP formalized this trade-off using flow-to-batch assignments, routed link load, and a worst-case inference-time surrogate. It first minimizes modeled inference time, then minimizes active batches without degrading the primary optimum. In an illustrative hotspot, the resulting schedule reduced normalized peak utilization from 1.5 to 0.5 and modeled inference time from 4.75 to 2.25, while increasing estimated migration time from 1.55 to 1.65 and using three batches. The purpose of the model was explanatory: it established that the apparent contradiction between "more batches" and "faster serving" disappears when the objectives are separated. The rank-disjoint rule is an online engineering choice rather than a constraint of the updated ILP.

The deployed scheduler translates that insight into deterministic first-fit edge coloring. Remote migrations are coalesced by ordered rank pair, and every batch forms a matching in the rank communication graph. Independent pairs retain concurrency, but no rank participates in multiple directed flows within a batch. All layer schedules are constructed in one cycle-wide CPU call, while expert weights remain transferred and activated layer by layer. The implementation is opt-in, async-only, and preserves the legacy path as a safe fallback.

On the evaluated four-node vLLM system, enabling batching improved output throughput by 2.28%, reduced TTFT P50/P99 by 1.18%/1.30%, reduced TPOT P50/P99 by 1.94%/0.87%, and reduced E2E P50/P99 by 3.18%/1.50%. Head-node NIC P99 fell by 10.24%. Profiling measured 34 scheduling calls for all 48 layers per rank, with 302.081 ms as the largest cumulative rank cost. That cost was 0.285% of the independently observed

105.932 s serving-time reduction. The code was merged upstream into vLLM in pull request #52641.

The work also has clear limitations. It reports a single A/B pair on one cluster, uses a synthetic random workload, samples NIC traffic on only the head node, and evaluates one migration backend. Accordingly, the result is an engineering case study rather than a universal throughput claim.

Several extensions follow naturally:

- repeat and randomize A/B runs on server-class GPUs and RDMA networks, reporting confidence intervals and per-node network counters;
- relax the one-flow-per-rank limit using observed foreground load or available bandwidth rather than fixing it statically;
- incorporate topology, heterogeneous expert sizes, and multiple NICs while retaining a deterministic fallback;
- evaluate synchronization semantics and benefit across every supported communicator backend;
- coordinate placement and migration scheduling so a placement policy can account for reconfiguration cost without coupling model routing to a complex online optimizer; and
- expose migration progress and interference metrics for automatic operator tuning.

The broader lesson is that online model reconfiguration should be evaluated as part of serving, not as an isolated data-transfer benchmark. A migration that finishes later can still be the better systems choice when it protects the latency and throughput of user requests.

Bibliography

[1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee. Taming the throughput–latency trade-off in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 117–134, 2024. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[2] W. Cao. [eplb] add contention-aware expert migration batching. vLLM pull request 52641, 2026. URL https://github.com/vllm-project/vllm/pull/52641. Merged 3 October 2026.

[3] M. Chowdhury, Y. Zhong, and I. Stoica. Efficient coflow scheduling with Varys. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, pages 443–454, 2014. doi: 10.1145/2619239.2626315.

[4] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

[5] N. Du, Y. Huang, A. M. Dai, et al. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569, 2022. URL https://proceedings.mlr.press/v162/du22c.html.

[6] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL https://arxiv.org/abs/2101.03961.

[7] T. Gale, D. Narayanan, C. Young, and M. Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. In *Proceedings of Machine Learning and Systems*, volume 5, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/5a54f79333768effe7e8927bcccffe40-Abstract-mlsys2023.html.

[8] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In

Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, pages 120–134, 2022. doi: 10.1145/3503221.3508418.

[9] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[10] C. Hwang, W. Cui, Y. Xiong, et al. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems*, volume 5, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/5616d34cf8ff73942cfd5aa922842556-Abstract-mlsys2023.html.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[12] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020. URL https://arxiv.org/abs/2006.16668.

[13] J. Li, Y. Jiang, Y. Zhu, C. Wang, and H. Xu. Accelerating distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference*, pages 945–959, 2023. URL https://www.usenix.org/conference/atc23/presentation/li-jiamin.

[14] X. Nie, X. Miao, Z. Wang, Z. Yang, J. Xue, L. Ma, G. Cao, and B. Cui. FlexMoE: Scaling large-scale sparse pre-trained model training via dynamic device placement. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2023. doi: 10.1145/3588964.

[15] NVIDIA and NIXL Contributors. NVIDIA Inference Xfer Library: Design and api guide. NIXL project documentation, 2026. URL https://github.com/ai-dynamo/nixl/blob/main/docs/nixl.md. Accessed: 2026-10-18.

[16] NVIDIA Corporation. Nsight Systems user guide. NVIDIA Documentation, 2026. URL https://docs.nvidia.com/nsight-systems/UserGuide/. Accessed: 2026-10-18.

[17] OmniInfer Contributors. OmniInfer: Expert placement and migration implementation. OmniInfer source repository, 2026. URL https://github.com/omni-ai-npu/omni-infer. Accessed: 2026-10-18.

[18] Qwen Team. Qwen3-30B-A3B-Instruct-2507 model card. Hugging Face, 2025. URL

https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507. Accessed: 2026-10-18.

[19] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18332–18346, 2022. URL https://proceedings.mlr.press/v162/rajbhandari22a.html.

[20] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1701.06538.

[21] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 173–191, 2024. URL https://www.usenix.org/conference/osdi24/presentation/sun-biao.

[22] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[23] vLLM Project. EPLBConfig api documentation. vLLM Documentation, 2026. URL https://docs.vllm.ai/en/latest/api/vllm/config/. Accessed: 2026-10-18.

[24] vLLM Project. Expert parallel deployment. vLLM Documentation, 2026. URL https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/. Accessed: 2026-10-18.

[25] Z. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024. URL https://arxiv.org/abs/2408.15664.

[26] D. B. West. *Introduction to Graph Theory*. Prentice Hall, 2 edition, 2001.

[27] A. Yang, A. Li, B. Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

[28] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[29] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. COMET: Fine-grained computation-communication overlapping for mixture-of-experts. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/e27ea0cd50b798ff8942caf9203f0992-Abstract-Conference.html.

[30] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 193–210, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[31] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022. URL https://arxiv.org/abs/2202.08906.

A | Reproduction Commands

This appendix records the essential final commands. The archived `COMMANDS.md` files remain the authoritative machine-readable source.

A.1. Environment and server

```
export NCCL_IB_DISABLE=1 NCCL_SOCKET_FAMILY=AF_INET
export NCCL_SOCKET_IFNAME=eno2 GLOO_SOCKET_IFNAME=eno2
export UCX_TLS=all UCX_NET_DEVICES=eno2
export UCX_RCACHE_MAX_UNRELEASED=1024
export VLLM_EPLB_LOG_MIGRATION_STATS=0

MODEL=Qwen/Qwen3-30B-A3B-Instruct-2507
USE_BATCHING=true  # repeat with false
EPLB_CONFIG="{\"window_size\":50,\"step_interval\":50,\
\"num_redundant_experts\":32,\"log_balancedness\":false,\
\"use_async\":true,\"communicator\":\"nixl\",\
\"enable_migration_batching\":$USE_BATCHING}"

vllm serve "$MODEL" --dtype float16 --port 8000 \
  --gpu-memory-utilization 0.9 --max-model-len 4096 \
  --enable-prefix-caching --tensor-parallel-size 4 \
  --distributed-executor-backend ray --enable-expert-parallel \
  --enable-eplb --eplb-config "$EPLB_CONFIG" --enforce-eager
```

A.2. Warm-up and formal workload

```
vllm bench serve --backend vllm --model "$MODEL" --port 8000 \
  --dataset-name random --random-prefix-len 300 \
  --random-input-len 200 --random-output-len 300 \
  --num-prompts 8 --max-concurrency 8 --seed 123 \
  --temperature 0 --ignore-eos --disable-tqdm

vllm bench serve --backend vllm --model "$MODEL" --port 8000 \
  --dataset-name random --random-prefix-len 300 \
```

```
  --random-input-len 200 --random-output-len 300 \
  --num-prompts 500 --max-concurrency 32 --seed 0 \
  --temperature 0 --percentile-metrics ttft,tpot,e2el \
  --metric-percentiles 50,99 --ignore-eos --disable-tqdm \
  --save-result --result-dir "$RESULTS" \
  --result-filename bench.json
```

A.3. NVTX-only Nsight capture

On the head node, Ray was launched inside a named Nsight session:

```
nsys start --session-new=eplb_final_0 --sample=none \
  --cpuctxsw=none -o "$RESULTS/trace/rank0"

nsys launch --session=eplb_final_0 --trace=nvtx --wait=all \
  ray start --head --node-ip-address=10.31.0.89 \
  --port=6379 --num-gpus=1 --disable-usage-stats
```

Each worker used the analogous command with its own session, output filename, and node IP. The trace files should be opened with Nsight Systems 2024.5.1 or newer. Searching for `eplb: schedule migration batches` locates the cycle-wide scheduler ranges.

The explicit NVTX range was temporary profiling instrumentation. It was not enabled in the clean E2E measurements and was removed from the final production change.

A.4. Unit tests

The upstream contribution was validated with:

```
.venv/bin/python -m pytest \
  tests/distributed/test_eplb_migration_scheduler.py -q
```

The reported result was 12 passing tests. Selected pre-commit hooks were also run on every modified production and test file.

B | Metric Definitions and Supplementary Results

B.1. Latency definitions

For request j, let a_j be its submission time, f_j the arrival time of its first output token, z_j the completion time, and o_j its number of output tokens. Then

$$\mathrm{TTFT}_j = f_j - a_j, \tag{B.1}$$

$$\mathrm{E2E}_j = z_j - a_j, \tag{B.2}$$

$$\mathrm{TPOT}_j = \frac{z_j - f_j}{o_j - 1}, \qquad o_j > 1. \tag{B.3}$$

TTFT includes waiting and prefill; it is not a pure GPU prefill-kernel time. TPOT is an average over post-first-token intervals for one request. E2E includes all queueing, prefill, decoding, and response completion observed by the client.

B.2. Complete improvement table

Table B.1 preserves the complete numerical values used to calculate every relative improvement reported in the main text.

Table B.1: Final clean A/B values and descriptive relative improvements.

Metric	Batching off	Batching on	Improvement
Request throughput (req/s)	0.105128	0.107523	2.278%
Output throughput (tok/s)	31.5385	32.2569	2.278%
TTFT P50 (ms)	49180.71	48599.04	1.183%
TTFT P99 (ms)	75740.25	74756.17	1.299%
TPOT P50 (ms)	828.527	812.439	1.942%
TPOT P99 (ms)	1007.821	999.006	0.875%
E2E P50 (ms)	303462.69	293821.76	3.177%
E2E P99 (ms)	322184.88	317351.61	1.500%
Head NIC P99 (MB/s)	599.814	538.398	10.239%

Request throughput and output throughput have the same relative improvement because every completed request contains exactly 300 generated tokens. They are shown separately here for traceability to the raw benchmark output, but the main discussion reports only output throughput.

B.3. Artifact map

Table B.2: Final artifact locations relative to `benchmarks/eplb_rebalance`.

Path	Purpose
`results/serving_nixl_20260911_async_random500/`	Clean E2E A/B root.
`random_async_off/bench.json`	Unmodified benchmark output with batching disabled.
`random_async_on/bench.json`	Unmodified benchmark output with batching enabled.
`random_async_off/nic.tsv` and `random_async_on/nic.tsv`	One-second head-node RX and TX counter samples.
`e2e_summary.csv`, `nic_summary.csv`	Derived values used by tables and plots.
`results/serving_nixl_20260911_async_random500_profile/`	Profiling run root.

Path	Purpose
`trace/rank0.nsys-rep` through `rank3.nsys-rep`	NVTX-only Nsight reports, one per rank.
`profile_summary.csv`	Extracted scheduler call count, percentiles, and cumulative cost.
`COMMANDS.md`	Exact profiling commands and trace interpretation.

Intermediate experiment directories are intentionally absent from this map. They were useful during development but are not data sources for the final thesis results.

Acknowledgements

I would like to thank my advisor, Prof. Qiaolun Zhang, and my co-advisor, Prof. Massimo Tornatore, for their professional support and guidance throughout this work. I also thank the vLLM maintainers and reviewers whose detailed feedback helped turn an experimental idea into a robust upstream contribution.